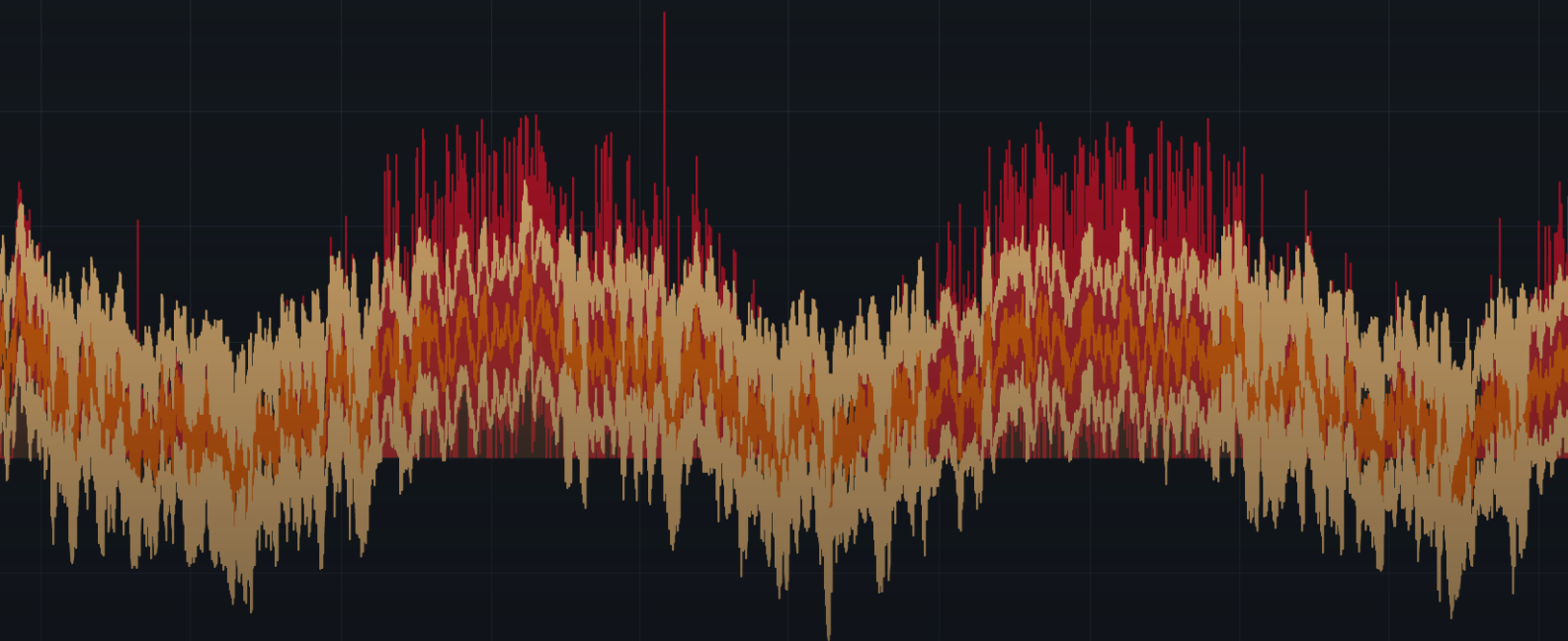




From a folder of data to a running pipeline.

An out-of-the-box data platform — database, orchestration, API and dashboards — that starts with one command and is meant to be taken apart and rebuilt around your own research.

What's in here

PART 1 · WHAT THIS IS AND WHY

The analysis that only runs on one laptop	3
Capabilities at a glance	4
How the pieces fit together	5
The worked example, end to end	6
Where else this fits	7
What this is not	8
Design decisions, and what they buy you	9

PART 2 · INSTALL AND FIRST RUN

Before you start	10
Three commands	11
Run the pipeline	12
Reading the run log	13
The payoff	14

PART 3 · THE SERVICES

Database	15
Roles and privileges	16
Migrations	17
Seeding	18
The REST interface	19
Querying without SQL	20
Dashboards	21
The orchestrator	22
The three example assets	23

PART 4 · MAKE IT YOURS

Recipe A — load your own data	24
Recipe B — add a table	25
Recipe C — write an asset	26
Recipe C — the transform	27
Recipe D — your own dashboard	28
Swapping the model	29

PART 5 · REFERENCE

Ports, variables, commands	30
When something goes wrong	31
Glossary	32

Who this is for

You analyse data competently — you know your statistics, you know your domain — but you have never had to stand up a database, and you would rather not become a systems administrator to share your work. Nothing here assumes you have.

What you'll have in 30 minutes

A time-series database holding 35,935 real measurements, a three-stage pipeline that fits a Bayesian model and writes its predictions back, and a dashboard showing measurement and prediction on one axis — all running on your machine, all reproducible by anyone you hand the folder to.

The analysis that only runs on one laptop

Most research data work is not lost to bad science. It is lost to the ordinary friction of files, folders and environments — and that friction is an infrastructure problem, not a competence problem.

The pattern is familiar enough to be invisible. Measurements arrive as files. The files accumulate in a folder, and the folder names start carrying meaning that exists nowhere else. Analysis happens in a notebook that is run top to bottom — usually. Results become images pasted into a slide deck. It works, and it keeps working, right up until one of these happens:

Five ways it stops working

- **New data arrives.** The whole sequence has to be re-run by hand, in the right order, and you have to remember what that order was.
- **Someone else needs to reproduce it.** The environment is your machine — your package versions, your paths, the one library you installed and forgot about.
- **Two analyses disagree.** Nothing recorded which version of the data each one used, so the disagreement cannot be resolved.
- **A collaborator wants to look, not compute.** Their only options are to be sent a static image or to be walked through running your code.
- **You move on.** The project leaves with you, because the working knowledge was never written down anywhere a successor could execute.

The conventional fix — a database, a scheduler, an API, a dashboard — is well understood. It is also four unfamiliar technologies, and learning them is not what you are paid to do.

What E-stack actually is

E-stack is that conventional fix, pre-assembled. Six containerised services — a time-series database, a schema versioning tool, a data loader, a REST interface, a dashboard server and a pipeline orchestrator — already wired together, already talking to each other, with a complete worked example running through all of them.

You start it with one command. It comes up populated, with a real pipeline you can run and a real dashboard you can look at. From there the intended use is not to admire it but to take it apart: replace the example data with yours, replace the example model with yours, and keep the plumbing.

The honest version of the pitch

E-stack does not make your analysis better. It makes your analysis *survive* — being re-run, being handed over, being checked by someone who was not in the room.

Capabilities at a glance

Six services, each solving one problem that would otherwise be solved badly by a folder of files.

Durable, queryable storage

A TimescaleDB database — PostgreSQL 18 with time-series extensions — holding your measurements in typed columns instead of parsed strings.

Ask questions of all your data at once, not one file at a time.

Versioned schema

Every change to the data model is a numbered, dated, checked-in migration applied by `sqitch` when the stack starts.

The shape of your data has a history you can read.

Repeatable pipelines

Dagster models your processing as *assets* with declared dependencies. Run one, and everything it needs runs first, in order.

No more remembering which cell to run second.

Data access without SQL

PostgREST turns the database into a REST API automatically. Filtering, ordering and paging happen in the URL.

Collaborators get data without getting database credentials.

Dashboards as code

Grafana panels are provisioned from JSON files in the repository. Edit in the browser, export back to a file, commit it.

A dashboard that survives the machine it was built on.

Reproducible environment

Every service is a container. Python dependencies are locked. The whole thing starts with `docker compose up`.

"Works on my machine" becomes "works on any machine".

What that adds up to

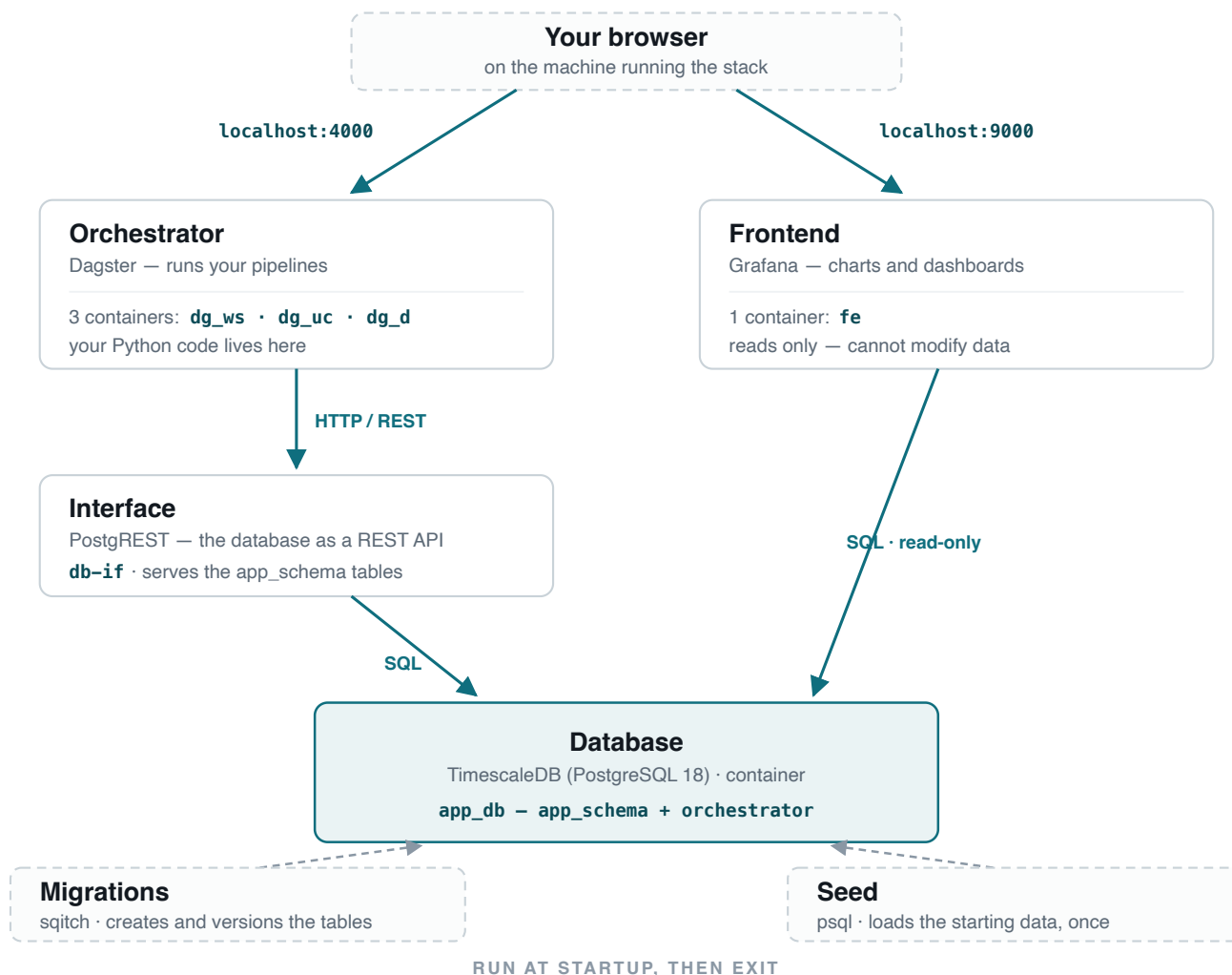
Each of these is unremarkable on its own — none of them is a new idea, and all of them are standard practice in software engineering. The point is that they are **already connected**. The database knows its own schema history. The orchestrator reaches the database through the API rather than around it. The dashboard reads with a role that cannot write. Wiring six tools together correctly is a week of work that has already been done, and the result is a starting point where the boring decisions are made and only the interesting ones are left to you.

Where each of these is demonstrated

CAPABILITY	SEE IT RUNNING	MAKE IT YOURS
Storage & schema	page 15	Recipe B — add a table, page 25
Versioned migrations	page 17	Recipe B, page 25
Pipelines	pages 12–13	Recipe C — write an asset, page 26
REST access	page 19	Querying without SQL, page 20
Dashboards	page 14	Recipe D, page 28
Reproducible environment	page 11	Reference, page 30

How the pieces fit together

Two things you open in a browser, one database underneath, and a strict separation between who may write and who may only read.



Reading the diagram

Solid teal lines are the live data path. Dashed grey lines are setup: those two containers do their job when the stack starts and then exit — they are not running services.

The asymmetry in the middle is deliberate. Your pipeline code reaches the data **through the REST interface**, not by opening a database connection. Grafana reaches it **directly, with a role that has only `SELECT`**. Page 9 explains why that is worth the extra hop.

The worked example, end to end

The stack ships with a complete, non-trivial pipeline already built. Understanding these five steps is most of what you need to build your own.

1 Data lands in the database

On first start, the seed service loads **35,935 hourly measurements** spanning 1 June 2017 to 7 July 2021 into `app_schema.energy_data`. Each row carries measured electrical `power`, ambient temperature `th_amb`, the previous day's ambient temperature `th_amb_prev_day`, and an `is_workday` flag.

2 The pipeline reads it back out — `raw_data`

The first Dagster asset requests the table through the REST interface and receives JSON. No SQL is written anywhere in the pipeline code.

```
# orchestrator/src/orchestrator/defs/assets.py
data_dict = await postgres.get_json_async(suburl="energy_data")
```

3 A model is fitted — `prediction_data`

The rows become a dataframe, and a Bayesian linear regression is specified in one line of formula syntax and fitted by MCMC sampling with two chains.

```
model = bmb.Model("power ~ th_amb + th_amb_prev_day + is_workday", data_df)
inf_data = model.fit(chains=2)
```

From the posterior predictive distribution the asset keeps three numbers per timestamp: the mean, and the 10th and 90th percentiles — an **80% credible band** rather than a bare point estimate.

4 Results go back where they came from — `ingest_prediction_data`

The three new columns are written back into the same rows, matched on their timestamp. Re-running the pipeline overwrites cleanly rather than duplicating.

```
await postgres.upsert_async(suburl="energy_data", json=pred_data_json,
                           on_conflict="time")
```

5 The dashboard shows both at once

Grafana plots measurement and prediction on one time axis — `power` in red, the predicted mean in orange, and the credible band as a lighter envelope around it. You can see where the model tracks reality and where it does not.

Why this example and not "hello world"

It is a full round trip — data leaves the database, is transformed by real statistical machinery, and returns to the same table where the dashboard can find it. Every piece of plumbing you would need for your own work is exercised at least once, so there is nothing left to guess at.

Where else this fits

The example is energy data, but nothing in the architecture is about energy. What the stack actually assumes is far narrower: measurements that carry a timestamp, and covariates you want to relate them to.

The four sketches below are illustrations of that shape — worked out to show what would and would not need to change. They are not descriptions of existing deployments.

Environmental sensor networks

Weather stations, air-quality monitors, soil or hydrological sensors — many devices reporting on a fixed interval, with gaps, drift and recalibration events that have to survive into the analysis.

What changes: the seed CSV, one migration adding your columns plus a station identifier, and the model formula. **What doesn't:** everything else.

Laboratory instrument runs

Spectra, chromatograms, plate readings — batches produced by an instrument that need identical preprocessing every time, and a record of which preprocessing produced which result.

What changes: one Dagster asset per processing stage — baseline correction, peak detection, quantification — chained by declared dependencies. **Why it helps:** re-running one stage automatically re-runs what depends on it.

Physiological and clinical monitoring

Continuous vitals or wearable telemetry, where the interesting question is usually a modelled quantity with an uncertainty attached, not the raw trace.

What maps directly: the prediction-plus-credible-band pattern in the example is exactly this shape. **Note:** the stack has no access-control layer — see page 8 before putting sensitive data in it.

Manufacturing and process telemetry

Machine or line sensors where you want an expected-behaviour envelope and an alert when the measurement leaves it.

What changes: add a Dagster schedule so the pipeline runs on a cadence instead of on a button press, and add a Grafana alert rule on the band. **What doesn't:** the data path.

The test for your own data

If you can describe your dataset as *"one row per point in time, with a few measured or categorical columns"*, the example maps onto it almost directly. If your data is not time series at all, most of the stack still applies — you would lose the TimescaleDB specifics and keep the database, migrations, orchestration, API and dashboards.

What this is not

A short and deliberately blunt list. Knowing where the boundaries are is what makes the rest of the claims in this manual usable.

It is local-first, and single-node

Everything runs as containers on one machine, orchestrated by Docker Compose. There is no clustering, no failover, no horizontal scaling and no backup strategy. It is designed for a workstation, a lab server, or a single virtual machine. Running it as shared infrastructure for an institution is a different project.

There is no authentication layer

PostgREST is configured with a single anonymous role, `interface_db_user`, so anyone who can reach the interface port can read and write the exposed tables. Grafana has a login; the REST API does not. **Treat the stack as trusted-network software.** Putting confidential or personal data in it means adding access control first.

The shipped credentials are placeholders

Every file in `secrets/examples/` contains the literal string `make-sure-to-change-and-seal-this` — including the Grafana administrator password. This is fine for a local first run and unacceptable anywhere else. Page 30 covers changing them.

The database is not yet a time-series database in the full sense

The image is TimescaleDB, so hypertables, continuous aggregates and compression are all *available* — but the example `energy_data` table is a plain PostgreSQL table with a unique constraint on `time`. It has not been converted into a hypertable. At 36,000 rows this makes no practical difference; at hundreds of millions it would, and converting is a one-line migration when you need it.

The example model is a demonstration

A linear regression on three predictors is there to prove the pipeline carries real statistical work end to end. It is not a validated load forecast, and no claim is made about its accuracy. Replacing it is the expected first move — page 29.

There is no test suite and no CI

The repository contains an empty `orchestrator/tests/` package and no continuous-integration configuration. Correctness of your own assets is currently your responsibility, and adding `pytest` to the `dev` dependency group is a reasonable first contribution.

Design decisions, and what they buy you

Six choices that look like extra ceremony from the outside. Each one is there because the obvious alternative fails in a specific, predictable way.

Everything is a container

Instead of: installing PostgreSQL, Grafana and Python on the machine.

Versions are pinned inside images, so the stack behaves identically on your laptop and your colleague's. Nothing is installed on the host. The cost is that you need Docker; the benefit is that you need nothing else.

Schema changes are versioned migrations

Instead of: running `CREATE TABLE` by hand when you need it.

Each change is a timestamped file in `db-migrations/src/deploy/`, listed in `sqitch.plan` and applied at startup. A colleague pulling the repository gets your schema without being told about it, and the history of how the data model evolved sits in version control beside the code depending on it.

The pipeline is a graph of assets, not a script

Instead of: a notebook, or `main.py` with functions called in order.

Dagster assets declare what they depend on. Asking for `ingest_prediction_data` runs `raw_data` and `prediction_data` first, automatically, in the right order — and records that it did, with timing, logs and metadata for every run. The execution order stops being something you have to remember.

Pipeline code talks to the database over HTTP

Instead of: a database connection string in your Python code.

This is the choice that most often looks like an unnecessary hop. It means your analysis code holds no database credentials, cannot accidentally issue a destructive statement, and is reachable by anything that speaks HTTP — a colleague's R script, a notebook, instrument software. One interface serves your pipeline and your collaborators.

Five database roles, not one

Instead of: connecting everything as the superuser.

Migrations own the tables; seeding, the interface and the orchestrator get exactly the privileges they need; and **Grafana holds a role with `SELECT` and nothing more**. A dashboard cannot corrupt data even if a panel query is wrong. The blast radius of a mistake is bounded by design rather than by care.

Dashboards are provisioned from files

Instead of: clicking a dashboard together and hoping it stays.

Panels are defined in JSON under `frontend/dashboards/` and loaded at startup. You can still edit freely in the browser, then export the result back to a file and commit it — so the dashboard is reproducible rather than resident in one container's storage.

Before you start

One piece of software, one setting, and about twenty minutes — most of which is Docker downloading images the first time.

What you need installed

Docker Desktop (macOS or Windows) or **Docker Engine with the Compose plugin** (Linux). That is the entire list. You do not need PostgreSQL, Python, Grafana or any of the other components on your machine — they exist only inside containers.

Check that it is working:

```
docker compose version
```

If this prints a version number you are ready. If it reports an unknown command, your Docker installation predates the Compose plugin and needs updating.

Raise Docker's memory limit before your first run

The example pipeline fits a Bayesian model by MCMC sampling over all 35,935 rows, in memory, inside a container. Docker Desktop's default allocation is often too small and the container is killed mid-run — which looks like an unexplained failure rather than an out-of-memory error.

In Docker Desktop, go to **Settings** → **Resources** and give it **at least 8 GB** of memory. On Linux the container uses host memory directly and no change is needed.

Get the repository

```
git clone git@github.com:vilisto-gmbh/e-stack.git
cd e-stack
```

How long the first run takes

STAGE	ROUGHLY	WHY
Building the images	5–15 min	Downloading base images and installing Python dependencies. Once only — later starts reuse the cache.
Database, migrations, seed	under 1 min	Creating the schema and copying 35,935 rows.
Running the example pipeline	1–3 min	MCMC sampling. This is genuine computation, not waiting.

Everything after the first start is fast: `docker compose up` on an already-built stack comes back in seconds.

Three commands

Two of them copy example configuration into place. The third starts everything.

1 Create your environment file

```
cp .env-example .env
```

This holds non-sensitive configuration — mostly which port each service uses. The defaults work; page 30 lists every variable if you need to change one because a port is already taken.

2 Create your secrets

```
cp secrets/examples/* secrets/
```

Six files, one password per service. Both `.env` and the copied secrets are excluded from version control, so they stay on your machine.

3 Start the stack

```
docker compose up
```

Leave this running — the logs from all services stream into this terminal, which is genuinely useful the first time. Stop it with `Ctrl-C`.

Change the passwords before this leaves your machine

Every copied secret contains the same placeholder, `make-sure-to-change-and-seal-this`. That includes `secrets/frontend_db_user_password.txt`, which is also the Grafana administrator password. Fine on a laptop; not fine anywhere reachable by anyone else.

What comes up

SERVICE	CONTAINER	WHAT IT IS	OPEN AT
Orchestrator	<code>dg_ws</code> <code>dg_uc</code> <code>dg_d</code>	Dagster — runs pipelines	<code>localhost:4000</code>
Frontend	<code>fe</code>	Grafana — dashboards	<code>localhost:9000</code>
Interface	<code>db-if</code>	PostgREST — the REST API	<code>localhost:3000</code>
Database	<code>db</code>	TimescaleDB	<code>localhost:5432</code> (SQL clients)
Migrations	<code>db-m</code>	sqitch — applies the schema	exits when done
Seed	<code>db-s</code>	psql — loads the sample data	exits when done

Migrations and seed exiting is *success*, not failure. They are one-shot jobs, and Compose holds the services that depend on them until they finish cleanly.

A note on the file name

The Compose file in this repository is `docker-compose.yaml`, spelled with `.yaml`. Plain `docker compose up` finds it either way; only an explicit `-f docker-compose.yml` would fail.

Run the pipeline

Open localhost:4000 — this is Dagster, where your data processing lives. Choose **Jobs** in the left sidebar, then **etl_pipeline_example**.

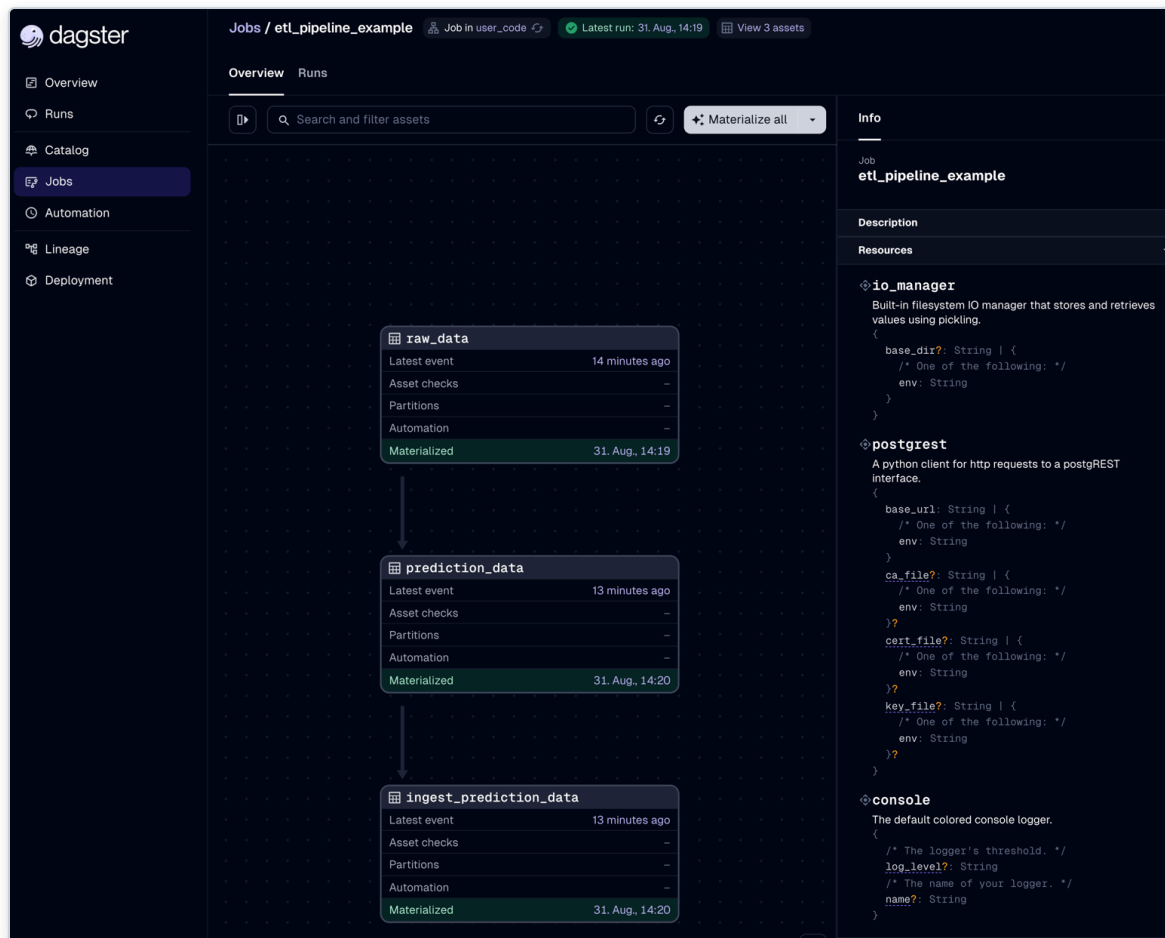


FIG. 1 The example pipeline as Dagster sees it. Three assets, each depending on the one above. The *Materialize all* button runs them in dependency order; the panel on the right lists the resources the job has access to, including the `postgrest` client it uses to reach the database.

What you are looking at

Each box is an **asset** — a named piece of data that your code knows how to produce. The arrows are dependencies that Dagster derives from the code itself, not from a configuration file you have to maintain.

Press **Materialize all** in the top right. All three assets run, in order, and turn green as they complete. Expect one to three minutes: the middle step is doing real MCMC sampling.

Why "materialize" rather than "run"

Dagster's unit of work is the data, not the script. You ask for an asset to *exist and be current*; Dagster works out which upstream steps must run to make that true. It is the same shift as asking for a built file instead of listing the compiler commands.

Reading the run log

Every run is recorded — each step, how long it took, and everything it logged. This is the part that makes a result defensible six months later.

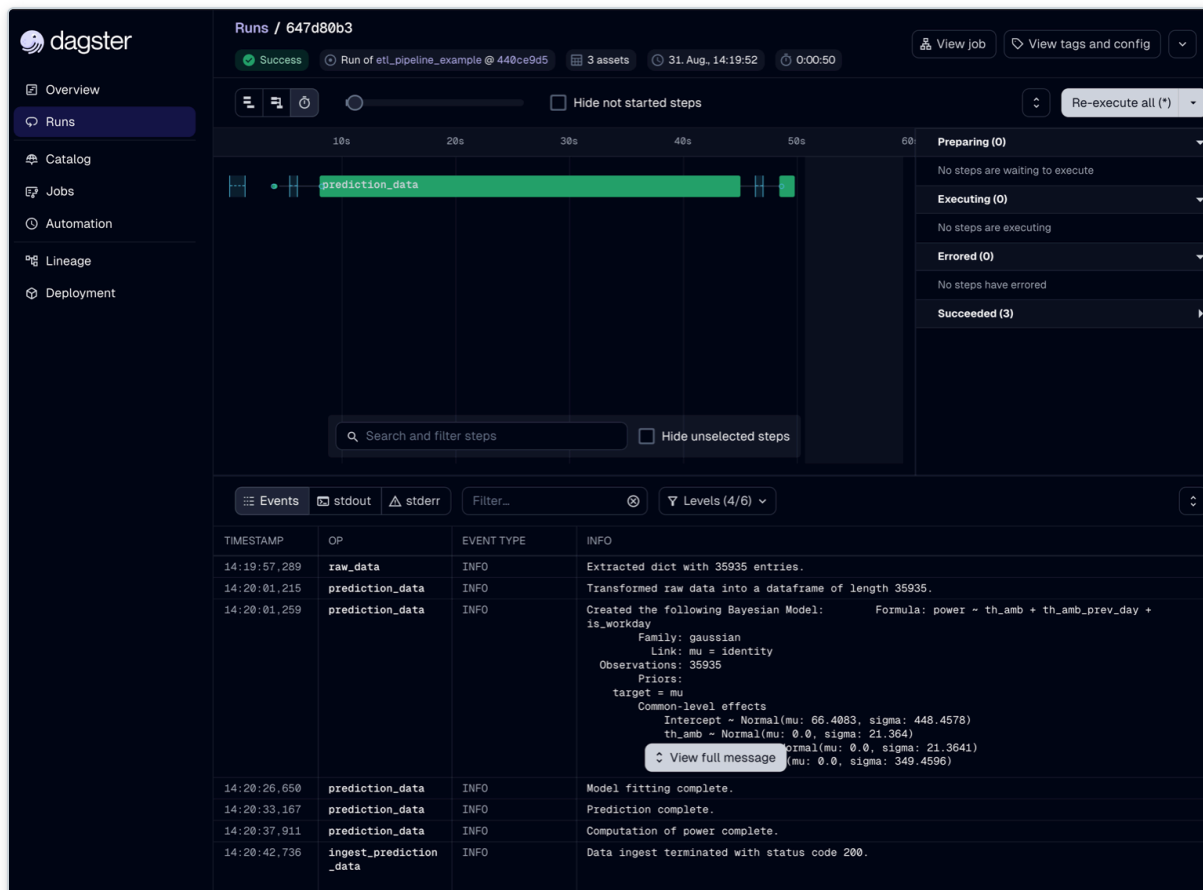


FIG. 2 A completed run: the timeline at the top shows where the 50 seconds went — almost all of it in `prediction_data` — and the event table below records what each step reported, including the full specification of the fitted model.

What the log is telling you

MESSAGE	MEANING
Extracted dict with 35935 entries.	The whole table came back through the REST interface.
Created the following Bayesian Model...	The model specification, printed in full — formula, family, link, observation count and the prior for every coefficient.
Model fitting complete.	MCMC sampling finished.
Prediction complete.	The posterior predictive distribution has been drawn.
Data ingest terminated with status code 200.	The predictions were accepted by the database.

That third entry is worth pausing on. It records the full formula with a Gaussian family and an identity link, over 35,935 observations, together with the priors actually used — which Bambi derives from the scale of your data unless you specify your own. Nobody has to reconstruct what was fitted. The run wrote it down.

The payoff

Open `localhost:9000`. Log in as `admin`, with the password in `secrets/frontend_db_user_password.txt`. Then go to **Dashboards** → **examples** → **example_dashboard**.

Do these two things first, or the chart will be empty

1. Run the pipeline before you look (page 12). Three of the four series on this chart are predictions, and they do not exist until the job has been materialized.
2. The sample data is from **2017 to 2021**. Grafana's default window is the last few hours, which contains nothing. The dashboard ships with the correct range saved — if you have changed it, set it back to roughly `2017-04-22` → `2021-07-07`.

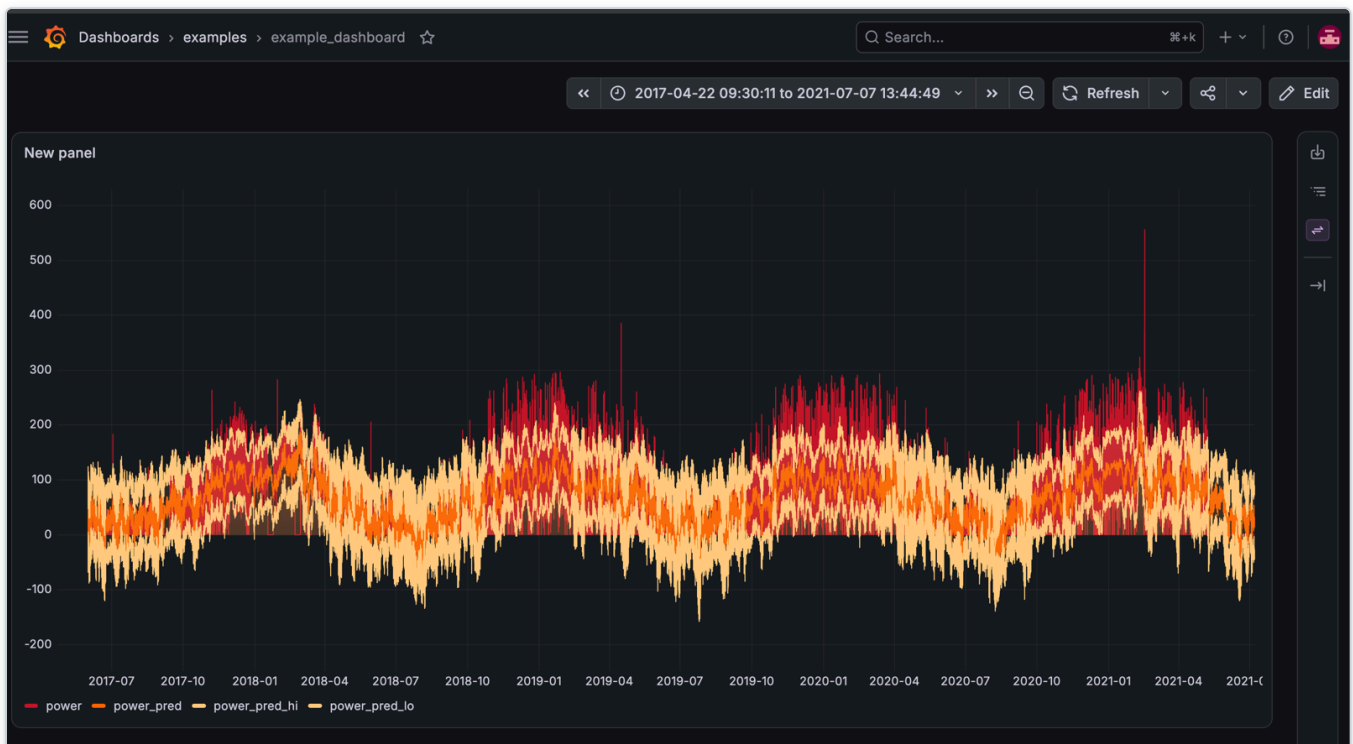


FIG. 3 Four years of hourly data on one axis. Measured `power` in red, the model's posterior mean `power_pred` in orange, and the 80% credible band between `power_pred_lo` and `power_pred_hi` in pale orange.

Reading the result

The seasonal structure is immediately visible — consumption climbing through each winter and falling away in summer — and so is the model's honesty about it. Where the band is wide the model is unsure; where the red measurement leaves the band entirely, something happened that ambient temperature and the workday flag cannot explain.

That is the useful property of carrying uncertainty through the pipeline rather than reporting a single fitted line: the chart shows you where to look.

At this point the loop is closed. Data went into a database, a pipeline read it, fitted a model and wrote its predictions back, and a dashboard rendered both together — reproducibly, on any machine with Docker. The rest of this manual is about replacing the parts of that loop with your own.

Database

One PostgreSQL 18 database with the TimescaleDB extension, holding both your data and the orchestrator's own record of what it has run.

Two schemas, on purpose

The database `app_db` is divided into two namespaces:

`app_schema` — **your data**. This is the only schema the REST interface exposes and the only one Grafana can read.

`orchestrator` — **Dagster's bookkeeping**. Run history, event logs and asset materialisation records. You never write to it directly; it is what lets the run log on page 13 survive a restart.

Keeping them apart means the orchestrator's growing history never gets confused with your measurements, and a query against your data cannot accidentally return Dagster's internals.

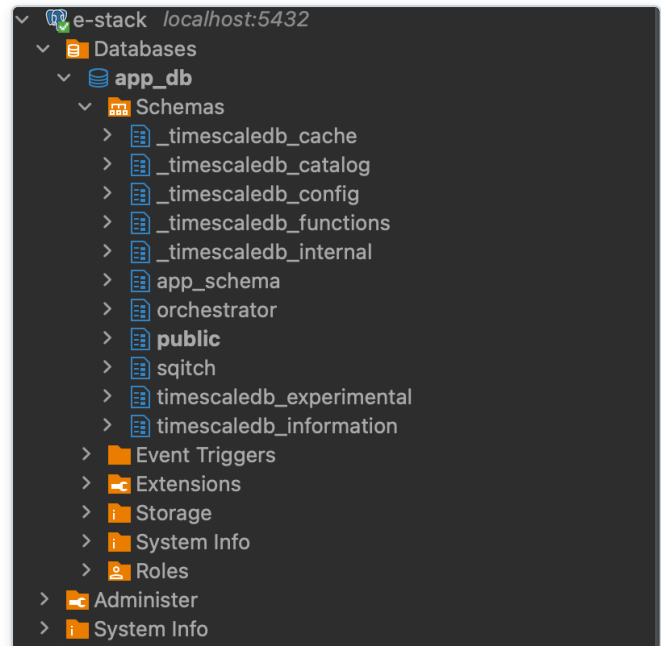


FIG. 4 `app_db` as any SQL client sees it. `app_schema` and `orchestrator` are yours; the `_timescaledb_*` and `sqitch` schemas belong to the extension and the migration tool.

The example table

A single table is created by the one migration that ships with the stack:

```
CREATE TABLE app_schema.energy_data (  
  time          TIMESTAMP WITH TIME ZONE UNIQUE,  
  power         FLOAT,           -- measured  
  power_unit    VARCHAR,  
  energy        FLOAT,  
  energy_unit   VARCHAR,  
  th_amb        FLOAT,           -- ambient temperature  
  th_amb_prev_day FLOAT,  
  is_workday    BOOLEAN,  
  power_pred    FLOAT,           -- written by the pipeline  
  power_pred_lo FLOAT,  
  power_pred_hi FLOAT  
);
```

The `UNIQUE` constraint on `time` is doing real work: it is what allows the pipeline to write predictions back with an *upsert* keyed on the timestamp, so re-running the job updates existing rows instead of duplicating them.

Connect any SQL client — DBeaver, psql, DataGrip — to `localhost:5432`, database `app_db`, to browse this yourself.

Roles and privileges

Six database users instead of one. This is the least visible design decision in the stack and one of the most valuable.

ROLE	USED BY	CAN DO
<code>postgres</code>	Nothing, routinely	Superuser. Creates the schemas and the other roles at first startup, then steps back.
<code>migrations_db_user</code>	<code>db-m</code>	Owens both schemas and everything in them. The only role that changes the shape of the database.
<code>seed_db_user</code>	<code>db-s</code>	Full access to <code>app_schema</code> tables, so it can bulk-load starting data.
<code>interface_db_user</code>	<code>db-if</code>	Full access to <code>app_schema</code> tables. Everything arriving through the REST API acts as this role.
<code>frontend_db_user</code>	<code>fe</code>	SELECT only. Grafana cannot insert, update, delete or alter anything.
<code>orchestrator_db_user</code>	<code>dg_uc</code> <code>dg_ws</code> <code>dg_d</code>	Full access to the <code>orchestrator</code> schema — its own run history — and nothing in <code>app_schema</code> .

Why this is worth the extra setup

Every one of these roles is a fence around a specific mistake.

- **Grafana is read-only**, so a mistyped panel query can be wrong but cannot be destructive. Dashboards are frequently edited by the least technical person in the room; this makes that safe.
- **Dagster's bookkeeping cannot touch your measurements.** The orchestrator writes its run history as `orchestrator_db_user`, which has no privileges on `app_schema` at all — your data reaches the database only through the REST interface, deliberately.
- **Only migrations can change the schema.** Nothing else owns the tables, so the structure of your data cannot drift out from under version control.

Where the passwords live

Each role's password is a file in `secrets/`, mounted into the container that needs it as a Docker secret and read at startup. Passwords are never written into `docker-compose.yaml`, never passed as plain environment variables, and never committed — `secrets/` is excluded from version control apart from the examples directory.

All six still contain the same placeholder

`make-sure-to-change-and-seal-this`. Replace the contents of each file in `secrets/` with a real password, then restart the stack with `docker compose down --volumes` followed by `docker compose up` — the roles are created on first database initialisation, so the volume has to go for new passwords to take effect.

Migrations

Every change to the structure of your database is a file, in order, in version control. The tool is `sqitch`, and it runs automatically each time the stack starts.

What a migration is

Three files per change, under `db-migrations/src/`:

DIRECTORY	CONTAINS
<code>deploy/</code>	The SQL that makes the change — <code>CREATE TABLE</code> , <code>ALTER TABLE</code> , and so on.
<code>revert/</code>	The SQL that undoes it.
<code>verify/</code>	A check that the change actually landed.

The file `sqitch.plan` lists them in order and is the authoritative record of your schema's history:

```
%project=e-stack

20260729111320_example_1 2026-07-29T11:13:20Z tillmeinel <...> # adding energy_data table
```

Adding one

A helper script creates all three files and appends the plan entry:

```
cd db-migrations/src
sh addMigration.sh add_station_column "adding a station identifier"
```

Run it from inside `db-migrations/src`

The script writes to `./sqitch.plan`, `./deploy`, `./verify` and `./revert` using relative paths. Run it from the repository root and it will fail, or quietly create files in the wrong place.

Then open the new file in `deploy/` and write your SQL beneath the generated comment line.

Applying it

Migrations run on every `docker compose up`. To apply a new one against a stack that is already running, without restarting anything else:

```
docker compose up --no-deps db-migrations
```

Re-running when there is nothing new to deploy is safe and exits cleanly — the stack is designed to be restarted freely.

Seeding

A one-shot service that puts starting data into an empty database, then makes sure it never does so twice.

What it does

After the migrations have created the tables, the `db-s` container runs a single bulk copy and exits:

```
psql $DB_URI -c "\copy app_schema.energy_data(time, power, th_amb,
    th_amb_prev_day, is_workday)
    FROM '/src/data/sample_data.csv' DELIMITER ';' CSV HEADER;"
```

Three details matter if you are replacing the file with your own.

- **The delimiter is a semicolon**, not a comma, and the file has a header row.
- **The column list is explicit**. Only the five named columns are loaded; everything else in the table stays `NULL` until something fills it.
- **Order matters**. The columns in the CSV must appear in the same order as the list in the command.

The sentinel

After a successful load the script leaves an empty marker file in a Docker volume:

```
touch /temp_data/sentinel
```

On every later start it finds that file and skips the load. This is what makes `docker compose up` safe to run repeatedly — without it, every restart would append another 35,935 rows.

Loading fresh data

Because the sentinel lives in a volume, clearing it means clearing volumes:

```
docker compose down --volumes
docker compose up
```

This deletes more than the seed marker

`--volumes` removes *all* persisted state: every row in the database, Dagster's entire run history, and any dashboard edits you made in the Grafana UI but did not export to a file. Use plain `docker compose down` for an ordinary stop — that keeps everything.

For data that arrives repeatedly

Seeding is for the *initial* contents of an empty database. Data that keeps arriving belongs in a Dagster asset instead, where each load is recorded, repeatable and safely re-runnable — see Recipe C on page 26.

The REST interface

PostgREST reads your database schema and serves it as a REST API. There is no API code in this repository, because there is none to write.

How it works

The service inspects `app_schema` at startup and publishes one endpoint per table. Create a table in a migration, restart, and it is on the API — no route to register, no serialiser to define, no model class to keep in sync with the database.

```
# every table in app_schema becomes an endpoint
http://localhost:3000/energy_data
```

Two names for one service

Which address you use depends on where you are calling from, and this trips people up more than anything else in the stack.

CALLING FROM	USE
Your machine — browser, <code>curl</code> , a local notebook	<code>http://localhost:3000</code>
Inside a container — your Dagster assets	<code>http://db-if:3000</code>

`db-if` is the container's name on the internal Docker network. It does not resolve from your machine, and `localhost` inside a container means that container — not your laptop. The orchestrator builds the internal address from environment variables, so your asset code never has to spell it out.

What is exposed, and as whom

SETTING	VALUE
Schema served	<code>app_schema</code> — and only this one
Acting database role	<code>interface_db_user</code>
API port	<code>3000</code>
Health-check port	<code>3001</code> — used by Docker, not by you

Every request has the same privileges

There is no per-user authentication. Anyone who can reach port 3000 acts as `interface_db_user`, which can read *and write* every table in `app_schema`. On a laptop that is exactly what you want. On any shared network it is not — see page 8.

The quiet benefit

Because the interface speaks plain HTTP, anything can use it. A colleague working in R, a student in a Jupyter notebook, a piece of instrument software that can post to a URL — all reach the same data as your pipeline, without a database client and without credentials you would have to issue and later revoke.

Querying without SQL

Filtering, selecting, sorting and paging all happen in the URL. This page is most of what you will need day to day.

The building blocks

YOU WANT	ADD TO THE URL
Only some columns	<code>?select=time,power</code>
Rows above a value	<code>?power=gt.20</code>
A time window	<code>?time=gte.2020-01-01&time=lt.2021-01-01</code>
Sorted results	<code>?order=time.asc</code> or <code>time.desc</code>
Just the first thousand	<code>?limit=1000</code>
The next thousand	<code>?limit=1000&offset=1000</code>
Rows where a value exists	<code>?power_pred=not.is.null</code>
One of several values	<code>?is_workday=in.(true,false)</code>

Comparison operators: `eq` `neq` `gt` `gte` `lt` `lte` `like` `ilike` `is` `in`. Join conditions with `&` — they combine with AND.

A real request

Every workday reading from 2020 where the model predicted more than 100, newest first:

```
curl "http://localhost:3000/energy_data?select=time,power,power_pred\
&time=gte.2020-01-01&time=lt.2021-01-01&is_workday=is.true\
&power_pred=gt.100&order=time.desc&limit=500"
```

The same thing from a Dagster asset

The `postgrest` resource takes the query as a dictionary, so you never build URL strings by hand:

```
data = await postgrest.get_json_async(
    suburl="energy_data",
    params={"select": "time,power", "time": "gte.2020-01-01",
           "order": "time.asc", "limit": 1000},
)
```

Ask for the rows you need rather than fetching everything and discarding most of it. The example pipeline deliberately pulls the whole table — it is small, and it keeps the example readable — but real pipelines should narrow the request.

Writing data back

Use `upsert_async` with the column that identifies a row. Matching rows are updated in place; new ones are inserted.

```
await postgrest.upsert_async(suburl="energy_data",
                             json=records, on_conflict="time")
```

Dashboards

Grafana, with its database connection already configured and its example dashboard already loaded — both defined by files in the repository rather than by clicking.

What is set up for you

ITEM	DETAIL
Address	<code>localhost:9000</code>
Login	<code>admin</code> · password from <code>secrets/frontend_db_user_password.txt</code>
Data source	Named PostgreSQL , connected to <code>app_db</code> , read-only
Dashboard folder	examples , holding <code>example_dashboard</code>

Grafana is the one service that talks to the database directly in SQL rather than through the REST interface — it is a database client by design, and it holds a role that can only `SELECT`.

Panels are SQL

The example panel runs four separate queries and overlays them on one time axis:

```
SELECT time, power      FROM app_schema.energy_data;
SELECT time, power_pred FROM app_schema.energy_data;
SELECT time, power_pred_hi FROM app_schema.energy_data;
SELECT time, power_pred_lo FROM app_schema.energy_data;
```

Each becomes one series, and per-series colour overrides produce the red measurement and the orange prediction band you saw on page 14.

Edits made in the browser are not permanent

Grafana saves your changes into a Docker volume, which survives `docker compose down` but is destroyed by `docker compose down --volumes`. A dashboard that exists only in that volume is one command away from being lost, and cannot be shared with anyone.

Making a dashboard permanent

- 1 Edit the dashboard in the browser and save it, as normal.
- 2 Use the **export** control at the top right, then **export as code** → **copy to clipboard**.
- 3 Paste over the matching file in `frontend/dashboards/`, or add a new `.json` file there, and commit it.

New dashboard files are copied into the image at build time, so a new file needs `docker compose up --build frontend` to appear.

The orchestrator

Dagster is where your own code lives. It is worth ten minutes on its three core ideas, because everything you build here uses them.

Assets

An **asset** is a named piece of data your code knows how to produce — a table, a fitted model, a cleaned dataframe. You write a Python function, mark it with `@dg.asset`, and the function's name becomes the asset's name.

The shift from a script is that you describe *what should exist* rather than *what to run*. Dependencies are declared by taking an upstream asset as a function argument, and Dagster works out the order.

Jobs

A **job** is a named selection of assets you want to run together — what appears under *Jobs* in the sidebar. The example one is four lines:

```
etl_pipeline_example = dg.define_asset_job(
    name="etl_pipeline_example",
    selection=["raw_data", "prediction_data", "ingest_prediction_data"],
)
```

Resources

A **resource** is a shared connection to something outside your code. The stack provides one, `postgres`, already pointed at the interface. An asset receives it by naming it as a parameter — no configuration, no connection setup, no credentials in your code.

Where the files are

PATH	HOLDS
<code>defs/assets.py</code>	Your assets. This is the file you will edit most.
<code>defs/jobs.py</code>	Which assets are grouped into which job.
<code>defs/resources.py</code>	The PostgREST client and its configuration.
<code>defs/utils.py</code>	The HTTP layer underneath the client.
<code>pyproject.toml</code>	Python dependencies, managed with <code>uv</code> .

All paths are relative to `orchestrator/src/orchestrator/`. Anything you add under `defs/` is discovered automatically — there is no registry to update.

Three containers, one purpose

`dg_uc` runs your code, `dg_ws` serves the web interface, and `dg_d` handles scheduling in the background. Only the first one matters when you are writing assets; the split exists so that a mistake in your code cannot take down the interface you need to diagnose it.

The three example assets

All of `defs/assets.py`, in outline. Extract, transform, load — the template for almost anything you will build.

```
@dg.asset
async def raw_data(context, postgres: PostgresConnector) -> Any:
    # EXTRACT — pull the table through the REST interface
    data_dict = await postgres.get_json_async(suburl="energy_data")
    context.log.info(f"Extracted dict with {len(data_dict)} entries.")
    return data_dict

@dg.asset(deps=["raw_data"])
def prediction_data(context, raw_data: Any) -> pd.DataFrame:
    # TRANSFORM — fit the model, keep mean and credible band
    data_df = pd.DataFrame(raw_data)
    model = bmb.Model("power ~ th_amb + th_amb_prev_day + is_workday", data_df)
    inf_data = model.fit(chains=2)
    model.predict(inf_data, inplace=True, kind="response")

    post_pred = inf_data.posterior_predictive["power"]
    return data_df.assign(
        power_pred = post_pred.mean(dim=("chain", "draw")).values,
        power_pred_lo = post_pred.quantile(0.1, dim=("chain", "draw")).values,
        power_pred_hi = post_pred.quantile(0.9, dim=("chain", "draw")).values,
    )

@dg.asset(deps=["prediction_data"])
async def ingest_prediction_data(context, prediction_data, postgres) -> None:
    # LOAD — write the predictions back, matched on time
    await postgres.upsert_async(suburl="energy_data",
                               json=prediction_data.to_dict(orient="records"),
                               on_conflict="time")
```

	time	power	power_unit	energy	energy_unit	th_amb	th_amb_prev_day	is_workday	power_pred
1	2017-06-01 06:00:00.000 +0200	0.25	[NULL]	[NULL]	[NULL]	11.3	12.3	[v]	65.64
2	2017-06-01 07:00:00.000 +0200	24.321915	[NULL]	[NULL]	[NULL]	12.1	13.3	[v]	59.60
3	2017-06-01 08:00:00.000 +0200	0.25	[NULL]	[NULL]	[NULL]	14.2	14.8	[v]	49.0
4	2017-06-01 09:00:00.000 +0200	21.4032852	[NULL]	[NULL]	[NULL]	19	15.9	[v]	27.07
5	2017-06-01 10:00:00.000 +0200	0.9728766	[NULL]	[NULL]	[NULL]	20.2	16.6	[v]	22.66
6	2017-06-01 11:00:00.000 +0200	43.779447	[NULL]	[NULL]	[NULL]	20.1	18.5	[v]	15.8
7	2017-06-01 12:00:00.000 +0200	0.25	[NULL]	[NULL]	[NULL]	21.3	19.8	[v]	10.86
8	2017-06-01 13:00:00.000 +0200	0.25	[NULL]	[NULL]	[NULL]	21.9	20.5	[v]	7.02
9	2017-06-01 14:00:00.000 +0200	27.2405448	[NULL]	[NULL]	[NULL]	23.4	21.9	[v]	-2.30
10	2017-06-01 15:00:00.000 +0200	20.4304086	[NULL]	[NULL]	[NULL]	23.3	22.3	[v]	-1.65
11	2017-06-01 16:00:00.000 +0200	0.25	[NULL]	[NULL]	[NULL]	23.3	22	[v]	-0.79
12	2017-06-01 17:00:00.000 +0200	24.321915	[NULL]	[NULL]	[NULL]	24.4	22.9	[v]	-10.74
13	2017-06-01 18:00:00.000 +0200	0.25	[NULL]	[NULL]	[NULL]	24.3	23	[v]	-10.45
14	2017-06-01 19:00:00.000 +0200	21.4032852	[NULL]	[NULL]	[NULL]	24	21.7	[v]	-4.41
15	2017-06-01 20:00:00.000 +0200	0.25	[NULL]	[NULL]	[NULL]	22.5	21.4	[v]	3.67
16	2017-06-01 21:00:00.000 +0200	26.2676682	[NULL]	[NULL]	[NULL]	19.9	19.9	[v]	14.09
17	2017-06-01 22:00:00.000 +0200	21.4032852	[NULL]	[NULL]	[NULL]	18.1	16.9	[v]	28.0
18	2017-06-01 23:00:00.000 +0200	23.3490384	[NULL]	[NULL]	[NULL]	17.1	16	[v]	33.8

FIG. 5 The result in the database. `time`, `power`, `th_amb`, `th_amb_prev_day` and `is_workday` came from the seed file; the `power_pred` column on the right was written by the pipeline. `power_unit`, `energy` and `energy_unit` remain `NULL` — declared, never populated.

Recipe A — load your own data

The fastest way to see your own numbers in the stack. If your data has a timestamp and a few measured columns, this takes about fifteen minutes.

1 Shape your file

One row per point in time, a header row, and **semicolons** between fields. Booleans as `1` and `0`, timestamps as `YYYY-MM-DD HH:MM:SS`.

```
time;power;th_amb;th_amb_prev_day;is_workday
2017-06-01 04:00:00;0.25;11.3;12.3;1
```

2 Decide where it goes

If your measurements fit the existing columns — a timestamp, a measured value, one or two covariates — reuse `energy_data` and only the names in your head change. If you need different columns, do **Recipe B** first and come back.

3 Replace the file

```
cp ~/my_measurements.csv db-seed/data/sample_data.csv
```

Keeping the filename means nothing else has to change. To use your own name, update the path in `db-seed/seed_db.sh` as well.

4 Match the column list

In `db-seed/seed_db.sh`, the copy command names the columns explicitly. They must match your header, **in the same order**:

```
\copy app_schema.energy_data(time, power, th_amb, th_amb_prev_day, is_workday)
FROM '/src/data/sample_data.csv' DELIMITER ';' CSV HEADER;
```

5 Start clean

The seed only runs against a database that has not been seeded before, so the old marker and the old rows both have to go:

```
docker compose down --volumes
docker compose up --build
```

Check it landed

Open `http://localhost:3000/energy_data?limit=5` in a browser. Five of your rows as JSON means the load worked, the API sees it, and Grafana will too.

Recipe B — add a table

When your data does not fit the example columns. Every schema change goes through a migration, so it is recorded and repeatable.

1 Create the migration files

```
cd db-migrations/src
sh addMigration.sh add_spectra "adding the spectra table"
```

This creates three files named with today's timestamp — one each in `deploy/`, `revert/` and `verify/` — and appends a line to `sqitch.plan`. Run it from this directory, not the repository root.

2 Write the change

Open the new file in `deploy/` and add your SQL below the generated comment:

```
-- Deploy 20260901..._add_spectra.sql to pg --

CREATE TABLE app_schema.spectra (
  measured_at  TIMESTAMP WITH TIME ZONE UNIQUE,
  sample_id    VARCHAR,
  wavelength   FLOAT,
  intensity    FLOAT,
  baseline     FLOAT
);
```

Put a **UNIQUE** constraint on whatever identifies a row

Without it, `upsert_async(..., on_conflict="measured_at")` is rejected by the database, and re-running a pipeline appends duplicates instead of updating. This single line is the difference between a pipeline you can re-run and one you cannot.

3 Fill in revert and verify

```
-- revert/..._add_spectra.sql
DROP TABLE app_schema.spectra;

-- verify/..._add_spectra.sql
SELECT measured_at FROM app_schema.spectra WHERE FALSE;
```

The verify script should fail if the change did not land. Selecting from the new table with an always-false condition is the conventional way to check that it exists.

4 Apply it and publish it

```
docker compose up --no-deps db-migrations
docker compose restart db-if
```

The first command applies the migration to the running database. The second makes PostgREST re-read the schema, after which `http://localhost:3000/spectra` exists — with no API code written.

Recipe C — write an asset

Your own processing step, in `orchestrator/src/orchestrator/defs/assets.py`. Start by reading data in.

The shape of every asset

```
@dg.asset
async def my_asset_name(
    context: dg.AssetExecutionContext, # gives you context.log
    postgres: PostgresConnector,        # the database, by name
) -> Any:
    ...
    return something
```

Three rules cover most of it:

- **The function name is the asset name.** It is what appears in the graph and what you refer to elsewhere.
- **Naming a resource as a parameter connects it.** Ask for `postgres` and you are handed a working client.
- **Naming another asset as a parameter creates a dependency.** Dagster runs it first, automatically.

Reading your data in

```
@dg.asset
async def recent_spectra(context, postgres: PostgresConnector) -> Any:
    rows = await postgres.get_json_async(suburl="spectra",
        params={"measured_at": "gte.2026-01-01", "order": "measured_at.asc"})
    context.log.info(f"Fetchd {len(rows)} spectra.")
    return rows
```

Making Dagster see it

Your code is built into a container image, so edits need a rebuild:

```
docker compose up --build -d dagster_user_code
```

Then reload `localhost:4000`. If the change still does not appear, use **Reload definitions** under **Deployment**.

Adding it to a job

Assets appear in the catalogue on their own, but a job is what gives you a runnable pipeline — in `defs/jobs.py`:

```
spectra_pipeline = dg.define_asset_job(
    name="spectra_pipeline",
    selection=["recent_spectra", "corrected_spectra"],
)
```

Log generously

Every `context.log.info(...)` is stored with the run and stays readable months later.

Recipe C — transform and load

The second and third steps: do your actual work, then put the result somewhere it will outlive the run.

Transform

Depend on the previous asset by naming it as a parameter. This function is ordinary Python — whatever you would have written in a notebook belongs here, unchanged.

```
@dg.asset(deps=["recent_spectra"])
def corrected_spectra(context, recent_spectra: Any) -> pd.DataFrame:
    df = pd.DataFrame(recent_spectra)

    # --- your analysis goes here ---
    df["corrected"] = df["intensity"] - df["baseline"]

    context.add_output_metadata({
        "num_rows": len(df),
        "preview": dg.MetadataValue.md(df.head(3).to_markdown()),
    })
    return df
```

`add_output_metadata` attaches a summary to the run — row counts, a preview table, a fitted parameter — visible in the Dagster UI beside the asset.

Load

Write the result back to a table, matched on the column you gave a `UNIQUE` constraint in Recipe B:

```
@dg.asset(deps=["corrected_spectra"])
async def ingest_corrected(context, corrected_spectra: pd.DataFrame, postgres) -> None:
    rsp = await postgres.upsert_async(suburl="spectra",
                                     json=corrected_spectra.to_dict(orient="records"),
                                     on_conflict="measured_at")
    context.log.info(f"Ingest finished with status {rsp.status_code}.")
```

Every column you send must exist in the table

PostgreSQL rejects the whole request if the JSON contains a key with no matching column. If you added a computed column in Python, add it to the table in a migration first — or drop it from the dataframe before writing.

Running on a schedule

To have a pipeline run by itself rather than on a button press, add a schedule in `defs/jobs.py`:

```
daily_spectra = dg.ScheduleDefinition(
    job=spectra_pipeline,
    cron_schedule="0 3 * * *",  # 03:00 every day
)
```

The `dg.d` container runs schedules. New ones are created switched off — enable them under **Automation**.

Recipe D — your own dashboard

Build it by clicking, then commit it as a file so it belongs to the project rather than to your container.

1 Start from the example

Open **examples** → **example_dashboard**, and use **Save as** to make a copy. Starting from a working panel is faster than starting from an empty one, because the data source is already selected.

2 Point a panel at your data

Edit the panel, choose the **PostgreSQL** data source, switch the query editor to **code** mode, and write plain SQL. Return a time column and one value column per series:

```
SELECT measured_at AS time, corrected
FROM app_schema.spectra
ORDER BY measured_at;
```

Set the panel's format to **Time series**. Grafana uses the column named `time` for the x-axis, which is why the query aliases it.

3 Set a time range that contains your data

The single most common reason a new panel looks broken. Use the picker at the top right and choose a window your measurements actually fall in, then save the dashboard so the range is stored with it.

4 Export it into the repository

Use the export control at the top right → **export as code** → **copy to clipboard**. Save the JSON as a new file in `frontend/dashboards/`, then rebuild so it is provisioned from now on:

```
docker compose up --build -d frontend
```

Why bother exporting

A dashboard that only exists in the Grafana volume disappears with `docker compose down --volumes` and cannot be handed to anyone. A dashboard in `frontend/dashboards/` is version-controlled, reviewable, and appears automatically for every colleague who starts the stack.

Swapping the model

The statistics in the example occupy four lines. Replacing them with your own is the expected first move.

Where it lives

In `defs/assets.py`, inside `prediction_data`:

```
model    = bmb.Model("power ~ th_amb + th_amb_prev_day + is_workday", data_df)
inf_data = model.fit(chains=2)
model.predict(inf_data, inplace=True, kind="response")
```

Changing the formula

The string is [Wilkinson notation](#) — the same syntax as R's `lm` and `brms`. Everything left of `~` is what you are predicting; everything right of it are your predictors.

TO EXPRESS	WRITE
Different predictors	<code>power ~ th_amb + humidity</code>
An interaction	<code>power ~ th_amb * is_workday</code>
A grouping level per station	<code>power ~ th_amb + (1 station)</code>
Non-Gaussian outcomes	pass <code>family="poisson"</code> to <code>bmb.Model</code>

Sampling

`chains=2` is set low so the example finishes quickly. For real work use four, and expect the run to take proportionally longer:

```
inf_data = model.fit(chains=4, draws=2000, tune=1000)
```

Leaving Bayesian methods behind

Nothing in the stack requires Bambi. The asset returns a dataframe; how you produce it is entirely yours — `scikit-learn`, `statsmodels`, `SciPy`, or `arithmetic`. The only contract is that what you return has columns matching the table you write it to.

Adding a library

Dependencies are declared in `orchestrator/pyproject.toml` and locked with `uv`:

```
# add the package to [project] dependencies, then
cd orchestrator && uv lock
docker compose up --build -d dagster_user_code
```

Committing the updated `uv.lock` is what makes your environment reproducible for everyone else — it pins exact versions of every transitive dependency, not just the ones you named.

Ports, variables, commands

Where things are

SERVICE	ADDRESS
Dagster	localhost:4000
Grafana	localhost:9000
PostgREST	localhost:3000
Database	localhost:5432

Environment variables

In `.env`. Change a port here if it collides with something already running.

NAME	DEFAULT
POSTGRES_HOST	db
POSTGRES_PORT	5432
POSTREST_HOST	db-if
PGRST_SERVER_PORT	3000
PGRST_ADMIN_SERVER_PORT	3001
GRAFANA_PORT	9000
DAGSTER_WS_PORT	4000
DAGSTER_UC_PORT	4001

The two `_HOST` values are container names on the internal network. They are not reachable from your machine and should not be changed.

Secrets

One password per file in `secrets/`. All six ship with the same placeholder.

- `postgres_db_user_password.txt`
- `migrations_db_user_password.txt`
- `seed_db_user_password.txt`
- `interface_db_user_password.txt`
- `frontend_db_user_password.txt` — also the Grafana admin password
- `orchestrator_db_user_password.txt`

Commands

Everyday

```
docker compose up      # start
docker compose down    # stop, keep data
docker compose ps      # what runs
docker compose logs -f db
```

After changing code

```
# Python assets
docker compose up --build -d dagster_user_code

# a dashboard JSON file
docker compose up --build -d frontend

# a new migration
docker compose up --no-deps db-migrations
docker compose restart db-if
```

Starting over

```
# delete ALL data, history, dashboards
docker compose down --volumes

# full rebuild from scratch
docker compose up --force-recreate \
  --renew-anon-volumes \
  --remove-orphans --build
```

When something goes wrong

The failures people actually hit, in roughly the order they hit them.

Starting up

WHAT YOU SEE	WHAT TO DO
Compose exits complaining about a missing env file	You skipped step 1. Run <code>cp .env-example .env</code> .
A service fails reading <code>/run/secrets/...</code>	You skipped step 2. Run <code>cp secrets/examples/* secrets/</code> .
<code>port is already allocated</code>	Something else on your machine holds that port. Change the matching value in <code>.env</code> and start again.
Services hang waiting on <code>db</code>	Normal on a first run while the database initialises. If it lasts more than a couple of minutes, check <code>docker compose logs db</code> .

Running the pipeline

WHAT YOU SEE	WHAT TO DO
<code>prediction_data</code> dies with no clear error, or the container restarts	Almost always out of memory. Give Docker at least 8 GB — page 10.
Your code changes are not showing up	The code is baked into an image. Run <code>docker compose up --build -d dagster_user_code</code> , then Reload definitions under Deployment .
A write fails with a PostgREST error about a column	Your JSON contains a key with no matching column. Add it in a migration, or drop it from the dataframe first.
An upsert is rejected on <code>on_conflict</code>	That column has no <code>UNIQUE</code> constraint. Add one in a migration — page 25.
Re-running duplicates rows instead of updating	Same cause as above.

Looking at data

WHAT YOU SEE	WHAT TO DO
The dashboard is empty	Two candidates. The time range does not cover your data — the sample is 2017–2021. Or the pipeline has not run, so the prediction columns are still <code>NULL</code> .
Only the red series appears	The pipeline has not been materialized yet. Page 12.
Your dashboard edits vanished	<code>down --volumes</code> was used. Export dashboards to <code>frontend/dashboards/</code> to make them permanent — page 28.
New data did not load	The seed runs once and leaves a marker. <code>docker compose down --volumes</code> , then up again — page 18.
A new table is missing from the API	PostgREST caches the schema. <code>docker compose restart db-if</code> .

Reading logs

For a specific service, `docker compose logs -f <service>` — for example `db` or `dagster_user_code`. For anything a pipeline did, the Dagster UI beats the terminal: **Runs** → the run → **Events**.

Glossary

Terms this manual uses that are standard in software engineering and rarely explained to anyone else.

Container

An isolated, pre-packaged copy of a program together with everything it needs to run. It cannot see your machine's software and your machine cannot see inside it, which is why nothing has to be installed.

Image

The template a container is started from — the built, frozen result of a `Dockerfile`. Change source code and you must rebuild the image for the change to take effect.

Volume

Storage that outlives a container. The database, Dagster's history and Grafana's edits live in volumes, which is why `docker compose down` keeps them and `--volumes` destroys them.

Compose

The tool that starts several containers together with the right network, ports and startup order, described in `docker-compose.yml`.

Healthcheck

A test a container runs on itself so Compose knows it is genuinely ready, not merely started. It is what prevents the migrations running before the database can accept connections.

Migration

One recorded, ordered change to the structure of a database, stored as a file in version control so the same change can be applied to any copy.

Schema

A named group of tables inside a database. This stack has two: `app_schema` for your data and `orchestrator` for Dagster's own records.

Asset

In Dagster, a named piece of data your code knows how to produce, together with what it depends on. The unit you ask for, rather than a script you run.

Materialize

To run whatever is needed to bring an asset up to date. Asking for the last asset in a chain runs the whole chain.

Upsert

A write that updates a row if it already exists and inserts it if it does not, decided by a column with a `UNIQUE` constraint. What makes a pipeline safe to re-run.

REST API

A way of reading and writing data over ordinary web requests. Here it means a URL per table, with filters written into the address.

Credible interval

A range within which the modelled quantity lies with a stated probability. The band on the example dashboard is an 80% interval: the model puts 80% of its belief inside it.

MCMC

Markov chain Monte Carlo — the sampling method used to fit the Bayesian model. It explores the space of plausible parameter values rather than solving for one best answer, which is why it produces intervals and why it takes minutes rather than seconds.

Where to go next

The `README.md` in the repository is the condensed operator's reference. The upstream documentation for each component — TimescaleDB, sqitch, PostgREST, Grafana and Dagster — covers everything this manual deliberately left out.

If something here is wrong

This manual describes the stack as it stands today. Where it disagrees with the code, the code is right — and the disagreement is worth reporting, because a manual that has drifted is worse than none.